Algorithm XXXX: pysib, a Python Package for Prediction-Error Identification of SISO Polynomial Models

DIEGO ECKHARD, Universidade Federal do Rio Grande do Sul, Brazil

`pysib` is an open-source Python package for parameter identification of discrete-time single-input single-output (SISO) systems described by linear polynomial input–output models. It implements the prediction-error method (PEM) for the ARX, output-error, ARMAX, and Box–Jenkins model structures; the Steiglitz–McBride iteration and filtered continuation variants for non-convex prediction-error criteria; and instrumental-variable and correlation estimators for ARX-structure models under weaker noise assumptions. All estimators share a common interface: they return a parameter vector and a five-polynomial dictionary with entries `A`, `B`, `C`, `D`, and `F`, so models from different routines can be passed to the same prediction and simulation functions. Estimators that reduce to linear algebra are implemented in Python using NumPy and SciPy. The PEM estimators for non-convex criteria follow a deliberately conservative strategy of many small parameter updates with analytic sensitivity recursions; its many cost, gradient, and sensitivity evaluations are handled by compiled C extensions with LAPACK routines for the Gauss–Newton step. Monte Carlo experiments show that Steiglitz–McBride and PEM estimates under the output-error structure are centered close to the true parameters under additive output noise, and that filtered continuation raises the empirical success rate of the output-error PEM estimator from 60% to 100% on a non-convex prediction-error problem.

CCS Concepts: • **Mathematics of computing** → **Mathematical software**; • **Computing methodologies** → *Simulation and modeling*.

Additional Key Words and Phrases: system identification, Python, prediction-error method, polynomial models, mathematical software

ACM Reference Format:
Diego Eckhard. 2026. Algorithm XXXX: pysib, a Python Package for Prediction-Error Identification of SISO Polynomial Models. *ACM Trans. Math. Softw.* 0, 0, Article 0 (2026), 15 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Identifying dynamic models from measured input–output records is a central problem in control, simulation, and signal processing. Among the available model classes, discrete-time polynomial input–output structures—ARX, ARMAX, output-error, and Box–Jenkins—remain widely used because they are interpretable, compact, and well supported by the prediction-error framework [Ljung 1999, 2010; Söderström and Stoica 1989]. Estimating such linear dynamic models from data requires solving a prediction-error minimization problem. For ARX structures this criterion is a convex quadratic least-squares problem, whereas for OE, ARMAX, and Box–Jenkins structures the same PEM framework generally leads to non-convex least-squares criteria whose numerical outcome also depends on initialization and the optimization trajectory.

Author's Contact Information: Diego Eckhard, Universidade Federal do Rio Grande do Sul, Porto Alegre, Brazil, diegoeck@ufrgs.br.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-7295/2026/0-ART0
https://doi.org/XXXXXXX.XXXXXXX

The MATLAB System Identification Toolbox and the SIB toolbox provide mature, feature-rich implementations of these classical estimators, but they are either proprietary or tied to a commercial platform [Eckhard 2026b; The MathWorks, Inc. 2026]. In the open Python ecosystem, the SIPPY package offers a broad collection of input–output and state-space routines [CPCLAB-UNIPI 2026], and the Python Control Systems Library supports control analysis and design workflows [Fuller et al. 2021], while SysIdentPy focuses on nonlinear autoregressive models with exogenous inputs [Lacerda et al. 2020]. A gap remains for an open-source Python package that is specifically designed for polynomial input–output model structures, with a specialized PEM optimization core and a unified model convention that allows seamless switching between estimators.

`pysib` fills this gap as a compact software package for SISO polynomial identification. It implements classical estimators for ARX, output-error, ARMAX, and Box–Jenkins structures, including least squares, Steiglitz–McBride, instrumental variables, correlation error minimization, PEM, and filtered continuation variants for non-convex PEM criteria. The routines are exposed through a small Python API: each estimator returns a parameter vector and a common five-polynomial model dictionary, and the same returned model can be passed to shared prediction and simulation routines. This convention is intended to make the package usable as a computational tool rather than as a collection of unrelated functions. Estimators that reduce to direct linear-algebra problems are written in Python with NumPy and SciPy; the PEM estimators for non-convex criteria use compiled C extensions that interface with LAPACK for the Newton step.

The package makes three contributions whose combination distinguishes it from existing Python tools. First, it is a focused, open-source, and well-tested toolbox for polynomial input–output structures, with a documented public API, a layered Python/C architecture, and a common model representation that lets users switch between ARX-type, OE, ARMAX, and Box–Jenkins models without changing the downstream evaluation code. Second, the PEM estimators for the OE, ARMAX, and BJ structures use a specialized optimization core that combines analytic sensitivity recursions with a deliberately conservative search based on many small parameter updates followed by Gauss–Newton refinement; the large number of cost, gradient, and sensitivity evaluations demanded by this strategy would be impractical in interpreted Python loops, so the core is implemented as compiled C extensions with LAPACK linear solvers. Third, the package provides open implementations of filtered continuation methods [Eckhard et al. 2013, 2017] for non-convex PEM criteria; these methods progressively relax data filters to drive the estimate toward the original PEM criterion, and their effect is evaluated below on a non-convex OE-structure PEM problem.

The remainder of the article is organized as follows. Section 2 defines the prediction-error problem and the four polynomial model structures. Section 3 describes the estimators, the specialized PEM optimization core, and the filtered continuation strategy. Section 4 describes the public API, the common model convention, and the software architecture. Section 5 presents Monte Carlo experiments under noisy data and a study of the filtered continuation strategy on a non-convex output-error PEM problem. Section 6 concludes the article.

2 Problem and Model Classes

2.1 The Identification Problem

Consider a discrete-time single-input single-output (SISO) dynamical system whose internal structure is unknown. The system is driven by an input $u(t) \in \mathbb{R}$ and produces a true output $y_0(t) \in \mathbb{R}$, where t indexes discrete-time samples. The relationship is dynamic: $y_0(t)$ depends on the current and past values of u, and possibly on past values of the output itself.

An identification experiment applies an input signal $u(t)$ to the system for N time steps. The output is not measured exactly; what is recorded is

$$y(t) = y_0(t) + v(t), \qquad t = 1, \dots, N, \tag{1}$$

where $v(t)$ is an additive disturbance. No specific structure is assumed for $v(t)$ at this stage.

The identification problem is: given the data record $\mathcal{D}_N = \{u(t),\ y(t)\}_{t=1}^{N}$, determine a parametric model whose one-step-ahead prediction $\hat{y}(t \mid t-1; \theta)$—a function of past inputs and outputs, indexed by a finite-dimensional parameter vector θ—makes the prediction errors $\varepsilon(t; \theta) = y(t) - \hat{y}(t \mid t-1; \theta)$ small in a well-defined sense. This prediction is distinct from the simulated output, which is the noise-free response obtained by propagating the plant model from the input alone. The choice of model structure and the criterion for "small" are the subjects of the following sections.

2.2 Model Parameterization

The data are described by the general polynomial model structure, a parameterized family of candidate relations between the input, the noise, and the measured output:

$$y(t) = \frac{B(q^{-1}; \theta)}{A(q^{-1}; \theta)\, F(q^{-1}; \theta)}\, u(t - n_k) + \frac{C(q^{-1}; \theta)}{A(q^{-1}; \theta)\, D(q^{-1}; \theta)}\, e(t), \tag{2}$$

where θ is the vector of free polynomial coefficients to be estimated, $e(t)$ is a zero-mean white-noise sequence, q^{-1} is the backward-shift operator ($q^{-1}x(t) = x(t-1)$), and $n_k \geq 0$ is the input delay in samples. The polynomials are monic and defined in non-negative powers of q^{-1}:

$$\begin{aligned}
A(q^{-1}; \theta) &= 1 + a_1 q^{-1} + \cdots + a_{n_a} q^{-n_a}, \\
B(q^{-1}; \theta) &= b_0 + b_1 q^{-1} + \cdots + b_{n_b-1} q^{-(n_b-1)}, \\
C(q^{-1}; \theta) &= 1 + c_1 q^{-1} + \cdots + c_{n_c} q^{-n_c}, \\
D(q^{-1}; \theta) &= 1 + d_1 q^{-1} + \cdots + d_{n_d} q^{-n_d}, \\
F(q^{-1}; \theta) &= 1 + f_1 q^{-1} + \cdots + f_{n_f} q^{-n_f}.
\end{aligned}$$

In this mathematical parametrization the input delay is carried explicitly by n_k, and the numerator order n_b counts only the free coefficients of B, not the delay. In the model dictionary returned by an estimator, however, the delay is stored by prepending n_k zeros to the `B` array. Thus, the conceptual model keeps n_k separate from $B(q^{-1})$, while the returned software representation encodes the same delay inside `m["B"]`. For a given model structure, θ contains the coefficients of the *free* polynomials only (those not fixed to unity). For example, for the BJ structure all five polynomials contribute free coefficients, so $\theta = [b_0, \dots, b_{n_b-1},\ c_1, \dots, c_{n_c},\ d_1, \dots, d_{n_d},\ f_1, \dots, f_{n_f}]^\top$; for ARX, only A and B are free, giving $\theta = [a_1, \dots, a_{n_a},\ b_0, \dots, b_{n_b-1}]^\top$. The dimension n_θ depends on the chosen orders and structure. The four standard structures are special cases of (2) obtained by fixing some polynomials to unity, as listed in Table 1. The plant transfer function and noise model are, respectively,

$$G(q^{-1}; \theta) = \frac{B(q^{-1}; \theta)}{A(q^{-1}; \theta)\, F(q^{-1}; \theta)}, \qquad H(q^{-1}; \theta) = \frac{C(q^{-1}; \theta)}{A(q^{-1}; \theta)\, D(q^{-1}; \theta)}, \tag{3}$$

so (2) reads compactly as $y(t) = G(q^{-1}; \theta)\, u(t - n_k) + H(q^{-1}; \theta)\, e(t)$. At the true parameter vector θ_0 this gives $y_0(t) = G(q^{-1}; \theta_0)\, u(t - n_k)$ and $v(t) = H(q^{-1}; \theta_0)\, e(t)$, making precise the decomposition (1) of Section 2.1. A key structural distinction separates the four models: in ARX and ARMAX, the plant denominator and the noise model share the same polynomial A, so the dynamics of the noise and the plant are coupled; in OE and Box–Jenkins, the plant denominator F and the noise model C/D are independent, allowing the two components to be parameterized separately.

Table 1. Polynomial model structures supported by `pysib`. A free polynomial is estimated from data; a fixed polynomial is set to unity.

Structure	A	B	C	D	F	Noise model H
ARX	free	free	1	1	1	$1/A$
ARMAX	free	free	free	1	1	C/A
OE	1	free	1	1	free	1
BJ	1	free	free	free	free	C/D

The one-step-ahead predictor associated with the general structure (2) is [Ljung 1999]

$$\hat{y}(t \mid t-1;\theta) = y(t) + H(q^{-1};\theta)^{-1}\big[G(q^{-1};\theta)\,u(t-n_k) - y(t)\big], \tag{4}$$

where $H(q^{-1};\theta)^{-1} = A(q^{-1};\theta)D(q^{-1};\theta)/C(q^{-1};\theta)$. Substituting the polynomial assignments of Table 1 into (4) yields the structure-specific predictor. For the ARX structure,

$$\hat{y}(t \mid t-1;\theta) = \sum_{j=0}^{n_b-1} b_j\,u(t-n_k-j) - \sum_{i=1}^{n_a} a_i\,y(t-i), \tag{5}$$

which is linear in θ. For the OE, ARMAX, and Box–Jenkins structures:

$$\hat{y}(t \mid t-1;\theta) = \frac{B(q^{-1};\theta)}{F(q^{-1};\theta)}\,u(t-n_k), \qquad \text{OE}, \tag{6}$$

$$\hat{y}(t \mid t-1;\theta) = \frac{B(q^{-1};\theta)}{C(q^{-1};\theta)}\,u(t-n_k) + \frac{C(q^{-1};\theta)-A(q^{-1};\theta)}{C(q^{-1};\theta)}\,y(t), \qquad \text{ARMAX}, \tag{7}$$

$$\hat{y}(t \mid t-1;\theta) = y(t) + \frac{D(q^{-1};\theta)}{C(q^{-1};\theta)}\left[\frac{B(q^{-1};\theta)}{F(q^{-1};\theta)}\,u(t-n_k) - y(t)\right], \qquad \text{BJ}. \tag{8}$$

Thus, for Box–Jenkins models, prediction is evaluated by filtering the plant-output mismatch through the inverse noise model $D(q^{-1};\theta)/C(q^{-1};\theta)$ and adding the result to the measured output.

2.3 Parameter Estimation

The prediction-error method (PEM) estimates θ by minimizing the mean squared prediction error [Ljung 1999; Söderström and Stoica 1989]

$$V(\theta) = \frac{1}{N}\sum_{t=1}^{N} \varepsilon(t;\theta)^2, \tag{9}$$

where $\varepsilon(t;\theta) = y(t) - \hat{y}(t \mid t-1;\theta)$ is the one-step-ahead prediction error. For the ARX predictor (5), writing $\hat{y}(t \mid t-1;\theta) = \varphi(t)^{\top}\theta$ with the regression vector

$$\varphi(t) = \big[-y(t-1),\ldots,-y(t-n_a),\ u(t-n_k),\ldots,u(t-n_k-n_b+1)\big]^{\top}, \tag{10}$$

and stacking N rows into $\Phi \in \mathbb{R}^{N\times(n_a+n_b)}$ gives $V(\theta) \propto \|\Phi\theta - y\|^2$, whose minimum is the ordinary least-squares (OLS) solution $\hat{\theta} = (\Phi^{\top}\Phi)^{-1}\Phi^{\top}y$.

The gradient of (9) with respect to θ is determined by the *sensitivity matrix* $\Psi(\theta) \in \mathbb{R}^{N\times n_\theta}$, whose j-th column collects

$$\psi_j(t;\theta) = \frac{\partial \hat{y}(t \mid t-1;\theta)}{\partial \theta_j}. \tag{11}$$

Since $\varepsilon(t;\theta) = y(t) - \hat{y}(t \mid t-1;\theta)$, the chain rule gives

$$\frac{\partial V}{\partial \theta} = -\frac{2}{N}\,\Psi(\theta)^{\top}\varepsilon(\theta), \tag{12}$$

where $\varepsilon(\theta) = [\varepsilon(1;\theta), \ldots, \varepsilon(N;\theta)]^\top$. For the ARX structure, $\psi_j(t;\theta) = \varphi_j(t)$ (the j-th entry of the regression vector (10)), and (12) vanishes exactly at the OLS solution. For the OE, ARMAX, and Box–Jenkins structures, ψ_j requires filtering through $1/F(q^{-1})$ or $1/C(q^{-1})$—polynomials that depend on θ itself—making Ψ a nonlinear function of θ and (9) non-convex in general; the existence of multiple local minima for these linear model structures is well documented [Åström and Söderström 1974; Eckhard and Bazanella 2011; Eckhard et al. 2012; Söderström and Stoica 1982]. The Gauss–Newton approximation to the Hessian,

$$\frac{\partial^2 V}{\partial \theta^2} \approx \frac{2}{N}\,\Psi^\top \Psi, \tag{13}$$

retains only the outer-product term (dropping second-order sensitivity terms that vanish near the minimum). It is symmetric positive semidefinite by construction; when the sensitivity matrix Ψ has full column rank the Gram matrix $\Psi^\top\Psi$ becomes positive definite and the Gauss–Newton direction is well defined and points downhill. Under standard regularity conditions on the input and noise, global minimizers of the PEM criterion are consistent when the true system belongs to the model class [Ljung 1978, 1999]; the implemented PEM routines for non-convex criteria seek such minima through local optimization, so the numerical outcome also depends on initialization and the optimizer.

Several estimators in `pysib` avoid direct local optimization of a non-convex PEM criterion. They differ in their target model structure: SM targets the OE plant structure $G = B/F$, whereas IV and correlation target ARX-structure models.

Steiglitz–McBride (SM). The SM method [Steiglitz and McBride 1965] replaces the non-convex OE-structure PEM problem with a sequence of ARX least-squares problems solved on filtered data. The ARX step is computational, not the model structure ultimately reported: the returned estimate is an OE-structure plant model $G = B/F$, and no disturbance model H is identified. The iteration, its fixed-point properties, and the implementation choices are described in Section 3.1.2.

Instrumental Variables (IV). The IV method estimates an ARX model consistently even when the noise is not of the form $1/A(q^{-1};\theta)\,e(t)$, at the cost of requiring two independent experiments driven by the same input $u(t)$. Let $y_i(t) = y_0(t) + e_i(t)$, $i = 1, 2$, denote the noisy outputs, where $y_0(t)$ is the true plant output and $e_1(t)$, $e_2(t)$ are independent noise sequences. Define the regression matrix $\Phi_1 \in \mathbb{R}^{N\times(n_a+n_b)}$ with rows

$$\varphi_1(t)^\top = \big[-y_1(t-1), \ldots, -y_1(t-n_a),\ u(t-n_k), \ldots, u(t-n_k-n_b+1)\big],$$

and the instrument matrix $Z \in \mathbb{R}^{N\times(n_a+n_b)}$ with rows

$$\zeta(t)^\top = \big[-y_2(t-1), \ldots, -y_2(t-n_a),\ u(t-n_k), \ldots, u(t-n_k-n_b+1)\big],$$

where the lagged y_1 entries of φ_1 are replaced by the corresponding lagged y_2 entries. The IV estimate is

$$\hat{\theta} = (Z^\top \Phi_1)^{-1} Z^\top y_1. \tag{14}$$

Because Z shares the input-driven part of Φ_1 but $Z^\top e_1 \to 0$ as $N \to \infty$ (since e_2 and e_1 are independent), $\hat{\theta}$ is consistent [Söderström and Stoica 1989].

Correlation method. The correlation method finds θ by requiring the prediction error $\varepsilon(t;\theta)$ to be uncorrelated with an instrument $z(t)$ at M lags:

$$\frac{1}{N}\sum_t z(t-\tau)\,\varepsilon(t;\theta) = 0, \qquad \tau = 0, 1, \ldots, M-1. \tag{15}$$

Since $\varepsilon(t;\theta)$ is linear in θ for the ARX structure, (15) constitutes a system of M linear equations. For $M = n_a + n_b$ the system is exactly determined; for $M > n_a + n_b$ it is overdetermined and solved by least squares, which minimizes the sum of squared cross-correlations.

These estimators can be applied directly when their respective assumptions are appropriate. In addition, the ARX least-squares routine provides the initial parameter vector for every PEM estimator whose criterion is optimized iteratively in `pysib`: each such routine is warm-started from an ARX solution, so the optimizer begins from a meaningful point rather than an arbitrary initial guess.

3 Algorithms

3.1 Linear-Algebra-Based Estimators

3.1.1 ARX. The ARX estimator is the fundamental least-squares routine in the package and the warm-start for every iterative PEM routine. The ARX predictor (5) is linear in θ: $\hat{y}(t \mid t-1;\theta) = \varphi(t)^\top\theta$, where $\varphi(t)$ is the regression vector (10). Stacking N rows into the regression matrix $\Phi \in \mathbb{R}^{N\times n_\theta}$ and the outputs into $y = [y(1),\dots,y(N)]^\top \in \mathbb{R}^N$, the PEM criterion (9) reduces to

$$V(\theta) = \frac{1}{N}\,\|\Phi\theta - y\|^2,$$

which is a convex quadratic function of θ (the Hessian $\frac{2}{N}\Phi^\top\Phi$ is positive semidefinite). Differentiating and setting $\partial V/\partial\theta = 0$ yields the *normal equations*

$$\Phi^\top\Phi\,\hat{\theta} = \Phi^\top y.$$

The solution is unique if and only if Φ has full column rank, i.e., $\Phi^\top\Phi$ is invertible, in which case $\hat{\theta} = (\Phi^\top\Phi)^{-1}\Phi^\top y$ is the unique global minimum of $V(\theta)$. Full column rank is obtained in open-loop identification when the input $u(t)$ is *persistently exciting* of order n_θ: informally, the input must contain enough distinct frequency components to excite all modes of the regression matrix [Ljung 1999]. When this condition fails (e.g., a constant or insufficiently rich input), Φ is rank-deficient and $V(\theta)$ has infinitely many minimizers forming an affine subspace.

Computing $(\Phi^\top\Phi)^{-1}$ directly is numerically inadvisable even in the full-rank case: the condition number satisfies $\kappa(\Phi^\top\Phi) = \kappa(\Phi)^2$, so any ill-conditioning in Φ is squared, amplifying rounding errors. `pysib` instead calls `numpy.linalg.lstsq`, which invokes the LAPACK driver `dgelsd`. This routine solves the least-squares problem via the divide-and-conquer SVD of Φ directly, avoiding the normal equations entirely. When Φ is rank-deficient, `dgelsd` returns the minimum-norm solution among all minimizers of $V(\theta)$.

3.1.2 Steiglitz–McBride. The OE-structure predictor $\hat{y}(t \mid t-1;\theta) = B(q^{-1};\theta)/F(q^{-1};\theta)\,u(t-n_k)$ is nonlinear in the F coefficients, making the PEM criterion $V(\theta)$ non-convex even though the model is linear in the input and noise. The SM method approximates the OE-structure PEM solution by replacing the single non-convex problem with a sequence of auxiliary ARX least-squares problems.

At each iteration k, the current denominator estimate $\hat{F}^{(k)}(q^{-1};\theta)$ is used to prefilter both signals:

$$u_f(t) = \frac{u(t)}{\hat{F}^{(k)}(q^{-1};\theta)}, \qquad y_f(t) = \frac{y(t)}{\hat{F}^{(k)}(q^{-1};\theta)}.$$

An auxiliary ARX least-squares problem of orders (n_f, n_b) is then solved on the filtered pair (u_f, y_f) using the procedure of Section 3.1.1, and the new denominator estimate $\hat{F}^{(k+1)}$ is taken from the feedback polynomial of that auxiliary solution. When the additive disturbance $v(t)$ is white noise, a fixed point $\hat{F}^{(k+1)} = \hat{F}^{(k)}$ of this iteration satisfies the optimality conditions of the OE-structure PEM criterion; for colored disturbances, however, the fixed points are in general not stationary points of $V(\theta)$ and the SM estimate is biased [Ljung 1999; Stoica and Söderström 1981]. Convergence

to the global minimum is not guaranteed even in the white-noise case, and the result depends on the initialization. `pysib` initializes $\hat{F}^{(0)}$ from a plain ARX solution on the unfiltered data. The implementation uses the fixed iteration count $K = 100$: each iteration filters u and y by the current denominator estimate, calls the ARX routine on the filtered pair, and updates the OE denominator from the feedback polynomial of the auxiliary ARX result. The final SM model contains only the OE plant polynomials B and F; it does not estimate a separate noise model H.

3.1.3 Instrumental Variables. The IV estimator solves the linear system $(Z^\top \Phi_1)\,\hat{\theta} = Z^\top y_1$, where Φ_1 and Z are the regression and instrument matrices defined in Section 2.3. The system is solved by least squares via `numpy.linalg.lstsq`. Consistency requires the noise sequences of the two experiments to be mutually uncorrelated and uncorrelated with the input (Section 2.3). In `pysib`, the IV routine accepts a single pair (u, y_1) and a second output record y_2; when the two outputs are identical, the method reduces to ordinary least-squares ARX.

3.1.4 Correlation. The correlation estimator assembles the $M\times(n_a+n_b)$ matrix of cross-correlations between the instrument $z(t)$ and the lagged signals that appear in the ARX predictor, together with the corresponding right-hand side. The default instrument is $z = u$, which is valid in open-loop experiments where the input is uncorrelated with the noise. The linear system (15) is solved via `numpy.linalg.lstsq`, which in the overdetermined case $M > n_a + n_b$ yields the solution described in Section 2.3. The implementation follows the same pattern as ARX: the regressor matrix is assembled once and the result returned without iteration.

3.2 PEM Estimators for Non-Convex Criteria and Optimization Core

The OE, ARMAX, and Box–Jenkins routines estimate linear polynomial model structures by minimizing the PEM criterion $V(\theta)$ using the gradient (12) and the Gauss–Newton Hessian (13). Since the filters in these structures depend on the parameters, the resulting criteria are generally non-convex. For this reason, `pysib` uses a specialized optimization core rather than treating these estimation problems as generic black-box minimizations. Local convergence of maximum-likelihood and prediction-error iterations for ARMAX-type models is known to depend on problem-specific conditions [Goodwin et al. 2003], so the implementation emphasizes reproducible safeguards rather than a general convergence guarantee. The core combines structure-specific sensitivity recursions with a conservative search strategy based on many small parameter updates, followed by Gauss–Newton refinement. This design increases the number of error, gradient, and sensitivity evaluations, and the performance-critical routines are therefore implemented as compiled C extensions; the Newton system is solved through LAPACK. Each estimator is initialized from an ARX solution; the free coefficients of the noise-model polynomials C and D are initialized to zero, so that initially $C = D = 1$.

Structure-specific sensitivities. These PEM estimators use analytic, structure-specific sensitivity recursions rather than numerical differentiation. Separate C modules are used for OE, ARMAX, and Box–Jenkins because the free polynomials and the filters appearing in the sensitivity equations differ across structures. For the OE predictor $\hat{y}(t \mid t-1;\theta) = B(q^{-1})/F(q^{-1})\,u(t-n_k)$, for example, the sensitivities (11) are

$$\psi_{b_j}(t) = \frac{q^{-j}\,u(t-n_k)}{F(q^{-1})}, \tag{16}$$

$$\psi_{f_j}(t) = -\frac{q^{-j}\,\hat{y}(t \mid t-1;\theta)}{F(q^{-1})}. \tag{17}$$

For ARMAX the denominator filter is $1/C(q^{-1})$; for Box–Jenkins both $1/F(q^{-1})$ and $1/C(q^{-1})$ appear, together with the corresponding noise-model polynomials. Thus each C module implements the sensitivity recursions for its own polynomial structure.

Reuse of filtered sensitivity sequences. The C implementation also exploits the shift structure of polynomial models. Many sensitivity columns differ only by a delay. In these cases, the code first computes a filtered base sequence and then forms the corresponding columns by time shifts, instead of applying the same IIR filter separately for every coefficient. In the OE case, a single pass through $1/F(q^{-1})$ applied to the delayed input provides the base sequence for all numerator sensitivities, while a single pass applied to $-\hat{y}$ provides the base sequence for the denominator sensitivities; the columns associated with b_j and f_j are obtained by indexing these sequences with the appropriate lag. The same principle is used in the ARMAX and Box–Jenkins modules with their respective filters. This reuse is important because the optimizer evaluates the cost, gradient, and Gauss–Newton matrices many times along a single identification run.

Selective cost, gradient, and Hessian evaluation. The optimization core distinguishes between three levels of computation. When only a candidate parameter vector must be accepted or rejected, it evaluates the cost $V(\theta)$ without forming sensitivities. During the initial descent phase, it evaluates the cost and gradient but does not build the Gauss–Newton matrix. During the Newton phase, it evaluates the cost, gradient, and the matrix $\Psi^\top\Psi$ needed in (18). This separation avoids unnecessary gradient or Hessian assembly during the many small trial steps used by the optimizer. For scalar cost comparisons the C core internally uses the root-mean-square prediction error $\sqrt{V(\theta)}$, which does not change minimizers or accept/reject ordering because it is strictly increasing in V; the gradient and Gauss–Newton quantities retain the standard least-squares scaling and are unaffected by a constant factor in the cost.

Two-phase optimizer. Phase 1 is a conservative smoothed steepest-descent loop intended to stabilize the trajectory from the ARX warm start before local Newton refinement. At each of up to 10^4 gradient evaluations, the gradient $g = \partial V/\partial\theta$ (12) is accumulated by an exponential moving average, $d \leftarrow (4d + g)/5$, which damps iteration-to-iteration changes in the descent direction. The direction is normalized to unit length, separating the direction of motion from the step size, and the parameter vector is updated by $\theta \leftarrow \theta - \alpha\, d/\|d\|$. The step size α adapts slowly and multiplicatively: $\alpha \leftarrow 1.01\,\alpha$ when the cost does not increase and $\alpha \leftarrow 0.99\,\alpha$ on an increase, starting from $\alpha = 10^{-5}$. The phase terminates early when $\alpha < 10^{-7}$; in the implementation this test is applied once per block of 100 updates.

Phase 2 performs local Gauss–Newton refinement using the structured sensitivities computed by the C module. At each of 1000 iterations the Newton direction δ is obtained by solving

$$(\Psi^\top\Psi)\,\delta = -\Psi^\top\varepsilon \tag{18}$$

via the LAPACK routine `dposv`, which solves the Gauss–Newton normal system by Cholesky factorization under the assumption of positive definiteness. The candidate $\theta \leftarrow \theta - \alpha_k\,\delta$ uses a growing step $\alpha_k = k/1000$, ranging from 0.001 at the first iteration to approximately 1 at the last, so full Newton steps are approached gradually. If the candidate increases the cost or is non-finite, a bisection safeguard is applied up to ten times: $\theta^{\text{new}} \leftarrow (\theta^{\text{new}} + \theta)/2$. Phase 2 always performs the full 1000 iterations, with no early-termination test: the cost of the refinement phase is fixed by design, consistent with the conservative strategy of the core.

Algorithm 1 PEM optimization core for non-convex criteria

Require: initial parameter vector θ, cost J, gradient g, sensitivity matrix Ψ

initialize d as undefined, $\alpha \leftarrow 10^{-5}$
for up to 10^4 descent updates **do**
  compute $J(\theta)$ and $g(\theta)$
  if d is undefined **then**
    $d \leftarrow g(\theta)$
  else
    $d \leftarrow (4d + g(\theta))/5$
  end if
  $\theta_c \leftarrow \theta - \alpha d/\|d\|$
  if $J(\theta_c) \le J(\theta)$ **then**
    $\theta \leftarrow \theta_c$, $\alpha \leftarrow 1.01\alpha$
  else
    $\alpha \leftarrow 0.99\alpha$
  end if
  if $\alpha < 10^{-7}$ **then**
    stop descent phase
  end if
end for
for $k = 1, \ldots, 1000$ **do**
  compute $g(\theta)$, $\Psi(\theta)$, and $J(\theta)$
  solve $(\Psi^\top \Psi)\delta = -\Psi^\top \varepsilon$
  $\theta_c \leftarrow \theta - (k/1000)\delta$
  bisect θ_c toward θ until $J(\theta_c) \le J(\theta)$ or the trial budget is exhausted
  if $J(\theta_c) \le J(\theta)$ **then**
    $\theta \leftarrow \theta_c$
  end if
end for

return θ

3.3 Filtered Continuation Estimators

Non-convex prediction-error criteria may have multiple local minima, and even a conservative optimizer may converge to a nonglobal stationary point. The filtered estimators in `pysib` provide open implementations of the continuation and cost-function-shaping strategies developed for this setting [Eckhard et al. 2013, 2017]. At each continuation stage, filtering the input–output data modifies the shape of the cost function; progressively relaxing the filter then drives the estimate toward the original unfiltered criterion. Filtered variants are provided for the OE, ARMAX, and Box–Jenkins structures; in each case the final stage runs the corresponding unfiltered PEM estimator on the original criterion.

Stage sequence and warm-starting. The procedure runs nine filtered optimization stages followed by one final stage on the unfiltered data. Each stage calls the same PEM optimization core used by the corresponding unfiltered estimator, but on a filtered version of the input–output record. The solution returned by one stage is used as the initial condition for the next stage, so the continuation

path begins from a strongly shaped criterion and ends at the original PEM problem. The total computational cost is approximately ten times that of the corresponding unfiltered estimator.

Algorithm 2 Filtered continuation wrapper

Require: data u, y, model orders, PEM estimator E, filter poles $a_1, \ldots, a_9$
$u_f, y_f \leftarrow$ filter u, y with pole a_1
$\theta \leftarrow$ ARX-based estimate from u_f, y_f, converted to the parameter order of E
for $i = 1, \ldots, 9$ **do**
  $u_f, y_f \leftarrow$ filter u, y with pole a_i
  $\theta \leftarrow E(u_f, y_f, \text{orders}; \theta)$
end for
$\theta \leftarrow E(u, y, \text{orders}; \theta)$
construct the common model dictionary m

return (θ, m)

Filter schedule. For the OE, ARMAX, and Box–Jenkins structures, each stage uses a first-order IIR filter $(1 - a_i)/(1 - a_i q^{-1})$ with unit DC gain, whose pole is $a_i = 0.05^{1/\tau_i}$ with $\tau_i = 4(10 - i)$ for stages $i = 1, \ldots, 9$. The pole decreases from $a_1 \approx 0.920$ (aggressive low-pass, stage 1) to $a_9 \approx 0.473$ (mild low-pass, stage 9), progressively restoring high-frequency content. The final stage is the original unfiltered estimator applied to the same PEM criterion; the only difference between the standard and filtered routines is the initial condition—the standard routine starts from an ARX estimate, while the filtered routine starts from the solution of the previous filtered stage.

The numerical experiments in Section 5 focus on the OE structure because the theoretical analysis of cost-function shaping was developed specifically for the output-error case [Eckhard et al. 2013, 2017]. The same continuation strategy is implemented for ARMAX and Box–Jenkins in `pysib` under the expectation that the mechanism of progressively relaxing data filters should also improve convergence in those structures, although formal guarantees have not been established for them.

4 Software Interface and Organization

`pysib` is designed around a small public API rather than around separate estimator-specific model objects. The user supplies one-dimensional input and output arrays and the requested polynomial orders. Every estimation routine returns a pair $(\hat{\theta}, m)$: the vector $\hat{\theta}$ contains the free estimated parameters in the order used by the corresponding algorithm, whereas m is the model representation used by the rest of the package. This separation lets users inspect the numerical parameter vector when needed, while all prediction, simulation, and comparison routines operate on a common polynomial model.

The model m is a Python dictionary with keys `A`, `B`, `C`, `D`, and `F`. Each entry is a one-dimensional array containing the coefficients of the corresponding polynomial in Section 2.2; polynomials fixed by a model structure are stored as the scalar polynomial `[1]`. The same convention is used by the direct linear-algebra estimators, the iterative PEM estimators, and the filtered continuation variants. As a result, downstream code does not depend on which routine produced the model.

Table 2 summarizes the public routines. Each estimation routine takes the data arrays, the requested polynomial orders, and the delay n_k, consistently with Section 2.2; the order n_b counts the free numerator coefficients, not the leading zeros used internally to store the delay in the returned `B` polynomial. The exact signatures of all routines are documented in the user manual.

Table 2. Public routines in pysib.

Routine	Estimator or role
arx	Least squares for the ARX structure
sm	Steiglitz–McBride iteration for the OE structure
iv	Instrumental variables for the ARX structure
correlation	Correlation estimator for the ARX structure
oe	PEM for the OE structure
armax	PEM for the ARMAX structure
bj	PEM for the Box–Jenkins structure
oe_filtered	Filtered PEM for the OE structure
armax_filtered	Filtered PEM for the ARMAX structure
bj_filtered	Filtered PEM for the BJ structure
predict	One-step-ahead prediction
simulate	Noise-free plant simulation

The routines also have distinct modeling roles. The arx routine is the basic least-squares estimator for the ARX structure and the default warm start for the iterative PEM routines. sm provides a classical iterative approximation to PEM estimation of the OE plant structure $G = B/F$; internally it solves ARX least-squares subproblems, but its returned model uses the OE structure and does not include a fitted noise model. iv and correlation are alternative estimators for ARX-structure models in settings where the ordinary least-squares assumptions are unsuitable. The oe, armax, and bj routines call the PEM optimizer for non-convex criteria described in Section 3.2. The three filtered routines expose the continuation strategy of Section 3.3 through the same interface and return the same model dictionary.

A typical workflow estimates a model with one routine and evaluates it with the shared routines: a call to an estimator such as oe returns the pair $(\hat{\theta}, m)$, and the same dictionary m is then passed to predict and simulate, while its polynomial entries can be inspected directly. predict computes the one-step-ahead predictor $\hat{y}(t \mid t-1; \hat{\theta})$, using the full plant and disturbance model represented by G and H. In contrast, simulate computes the noise-free simulated output $\hat{y}_s(t; \hat{\theta})$ and uses only the deterministic transfer function G. This distinction is important in the numerical experiments below, where the relative simulation error is intended to measure the identified plant dynamics rather than the one-step prediction fit. Complete, runnable examples of every public routine are given in the user manual.

The package separates Python-level orchestration from performance-critical compiled routines. The ARX, IV, and correlation estimators, together with the auxiliary ARX least-squares subproblems used by SM, are implemented in Python using NumPy and SciPy. The PEM optimizers for non-convex criteria are compiled as three separate C extension modules (sib_oe_module.c, sib_armax_module.c, sib_bj_module.c), each providing the structure-specific sensitivity formulas and sharing the common two-phase optimizer from sib_optimize.c. The Newton step in Phase 2 links against LAPACK (dposv) without requiring an external system identification framework. The filtered variants are Python wrappers that sequence the filter stages and delegate each optimization to the corresponding C module.

This article describes the algorithms, interface, and high-level organization of the software. Installation instructions, dependency versions, complete API documentation, callable-procedure examples, and expected driver outputs are provided in the accompanying user manual, as required for the CALGO software component. Future updated versions of the software package will be made available via the source repository [Eckhard 2026a].

Table 3. Main software components of `pysib`.

Component	Implementation	Role
ARX, IV, Correlation	Python, NumPy, SciPy	ARX-structure estimators
SM	Python, NumPy, SciPy	OE-structure iteration via ARX subproblems
OE, ARMAX, BJ	C extension, LAPACK	PEM for non-convex criteria
Filtered variants	Python over C optimizer	Continuation strategy
Prediction and simulation	SciPy `lfilter`	Model evaluation

Table 4. Software artifact summary.

Item	Description
Language	Python package with compiled C extensions
Numerical libraries	NumPy, SciPy, and LAPACK
License	MIT
Documentation	User manual included with the CALGO artifact
Tests	Unit tests covering all public estimation routines
Experiments	Reproducible drivers for the numerical results in this article
Artifact contents	Source code, user manual, tests, experiment drivers, and expected outputs

Table 3 summarizes the main implementation components.

5 Numerical Results

The accompanying software artifact includes deterministic unit tests for noise-free exact recovery and for the polynomial convention of Section 2. These tests cover all public estimators—ARX, SM, IV, correlation, OE, ARMAX, BJ, and the filtered variants—on low-order systems with known parameters, and they exercise the common return convention used by the `predict` and `simulate` routines. They are part of the software validation and are not presented as scientific results. The numerical experiments reported here are reproducible drivers included with the artifact. They illustrate two end-to-end uses of the package: estimating models under noisy data and evaluating the effect of filtered continuation in a non-convex PEM problem. The purpose is to validate representative software workflows and numerical behavior, not to provide a general benchmark of all available system-identification packages.

5.1 Robustness under Noisy Data

The first experiment evaluates the SM and OE estimators under additive white output noise. The driver follows the same workflow exposed by the public API: generate an input–output record, call the estimators, receive the common model dictionary, and evaluate the returned models through the shared simulation routine. A second-order OE system with true polynomials $B(q^{-1}) = 1$, $n_k = 1$, and $F(q^{-1}) = 1 - 1.1q^{-1} + 0.2625q^{-2}$ is driven by the three-sine input $u(t) = \sin(2\pi t/17) + 0.7\sin(2\pi t/43) + 0.4\sin(2\pi t/97)$ of length $N = 500$. White Gaussian noise with standard deviation $\sigma_v = 0.1$ is added to the output, and $M = 500$ independent realizations are generated from a fixed pseudo-random seed, so all reported numbers are reproducible. SM and OE use orders $(n_b, n_f, n_k) = (1, 2, 1)$ matching the true plant. Table 5 reports the mean and sample deviation of the OE-structure parameter estimates. The SM and OE estimates are centered close to the true parameters, with deviations below 5×10^{-3}; for SM this behavior is consistent with the white-noise setting, under which the SM fixed points satisfy the OE-structure PEM optimality

Table 5. Mean and standard deviation of the OE-structure parameter estimates under noisy data ($M = 500$).

Estimator	b_0 (true 1)	f_1 (true -1.1)	f_2 (true 0.2625)
SM	1.0004 ± 0.0044	-1.0997 ± 0.0038	0.2623 ± 0.0033
OE	1.0003 ± 0.0044	-1.0998 ± 0.0038	0.2623 ± 0.0033

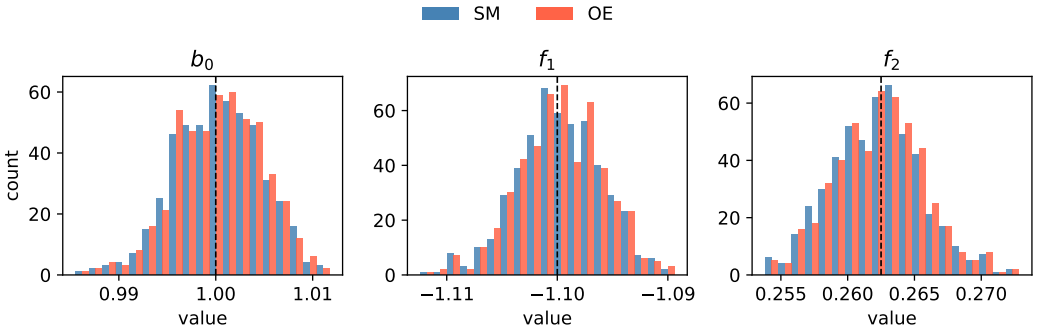


Fig. 1. Monte Carlo histograms of the OE-structure parameter estimates for SM and OE. Dashed vertical lines mark true values.

conditions (Section 3.1.2). Figure 1 shows the corresponding histograms per parameter. The driver, which additionally fits a deliberately mismatched second-order ARX model to the same records to illustrate the effect of an incorrect noise structure, is included in the software artifact together with its expected outputs.

5.2 Effect of Filtered Continuation

The second experiment tests whether the filtered continuation strategy improves the convergence of the OE estimator when the PEM criterion has multiple local minima. It exercises the filtered-estimator interface in the same way a user would use it: the standard and filtered routines return the same model representation and are evaluated by the same simulation-error computation. The true system is a third-order OE plant with $B(q^{-1}) = 1$, $n_k = 1$, and $F(q^{-1}) = 1 - 2.4q^{-1} + 1.91q^{-2} - 0.504q^{-3}$, corresponding to poles at 0.9, 0.8, and 0.7. The input is the three-sine signal $u(t) = \sin(2\pi t/18) + \sin(2\pi t/28) + \sin(2\pi t/61)$ of length $N = 1000$, and white Gaussian noise with $\sigma_v = 30$ is added to the output, yielding a signal-to-noise ratio of approximately -6 dB. A Monte Carlo of $M = 100$ independent realizations is performed from a fixed pseudo-random seed; both estimators use ARX-based initializations and the same optimization core (Section 3.2): the standard OE PEM routine proceeds directly to the unfiltered criterion, whereas the filtered variant first follows the filtered continuation path before its final unfiltered stage.

Quality is measured by the relative simulation error $\|y_0 - \hat{y}_s\|_2/\|y_0\|_2$ on the noise-free output, and a run is considered successful if this error is below 5%, a typical practical tolerance for model-based control applications. Table 6 reports the mean and standard deviation of the error for both routines, as well as the success rate. The standard OE estimator succeeds in 60% of the Monte Carlo runs (95% Wilson confidence interval $[50\%, 69\%]$), with a mean error of 5.7×10^{-2}. The filtered variant succeeds in 100% of the runs (95% Wilson confidence interval $[96\%, 100\%]$) and reduces the mean error to 2.2×10^{-2}. Figure 2 shows the error distribution and a boxplot of both estimators. The tenfold increase in computational cost due to the nine filtered stages is the price

Table 6. Relative simulation error of OE and OE filtered on a non-convex OE-structure PEM problem ($M = 100$).

Estimator	Mean error	Std error	Success rate ($< 5\%$)
OE	5.69×10^{-2}	4.37×10^{-2}	60%
OE filtered	2.17×10^{-2}	7.94×10^{-3}	100%

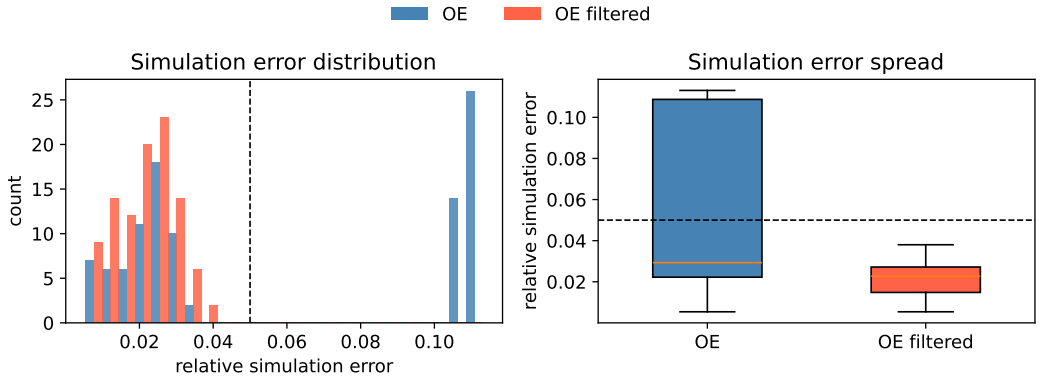


Fig. 2. Distribution of the relative simulation error for OE and OE filtered on the non-convex problem. Dashed line marks the 5% success threshold.

paid for achieving a 100% empirical success rate in this problem. The experiment is limited to OE because the cost-function-shaping theory motivating the filtered implementation was developed for output-error models; the same software pattern is provided for ARMAX and Box–Jenkins through the filtered routines listed in Table 2.

6 Conclusion

`pysib` is an open-source Python package for parameter identification of discrete-time SISO systems using classical polynomial input–output model structures. It provides a unified interface to estimators for ARX, output-error, ARMAX, and Box–Jenkins structures, including least-squares, Steiglitz–McBride, instrumental-variable, correlation, PEM, and filtered PEM routines. All estimation routines return the same kind of object: a free-parameter vector and a five-polynomial dictionary. This common convention lets users compare routines and evaluate models through shared prediction and simulation routines.

The package makes three main contributions. First, it fills a gap in the open Python ecosystem by delivering a focused, well-tested implementation of classical polynomial estimators with a compact API and a consistent model representation that lets users switch between model structures without changing the downstream evaluation code. Second, the PEM estimators for non-convex criteria use a specialized optimization core that combines analytic sensitivity recursions with a deliberately conservative many-step search; the core is implemented as compiled C extensions with LAPACK calls, orchestrated from Python. Third, the toolbox provides open implementations of filtered continuation methods [Eckhard et al. 2013, 2017] for non-convex PEM criteria; these methods progressively relax data filters to drive the estimate toward the original PEM criterion.

Numerical experiments show that under noisy data the SM and OE estimates are centered close to the true parameters, and that the filtered continuation strategy raises the success rate on a non-convex OE-structure PEM problem from 60% to 100% of the runs. The complete source code,

unit tests, user manual, and reproducible drivers are included in the accompanying CALGO software artifact.

Acknowledgments

This work was supported by CNPq/Brazil.

References

K. J. Åström and T. Söderström. 1974. Uniqueness of the maximum likelihood estimates of the parameters of an ARMA model. *IEEE Trans. Automat. Control* 19, 6 (1974), 769–773. doi:10.1109/TAC.1974.1100795

CPCLAB-UNIPI. 2026. SIPPY: Systems Identification Package for PYthon. Online. https://github.com/CPCLAB-UNIPI/SIPPY Accessed: 2026-05-25.

Diego Eckhard. 2026a. pysib source code repository. Online. https://github.com/diegoeck/pysib Accessed: 2026-07-03.

Diego Eckhard. 2026b. SIB – System Identification Toolbox. Online. https://professor.ufrgs.br/diego/software/slow-better Accessed: 2026-05-25.

D. Eckhard and A. S. Bazanella. 2011. On the global convergence of identification of output error models. *IFAC Proceedings Volumes* 44, 1 (2011), 9058–9063. doi:10.3182/20110828-6-IT-1002.03566

D. Eckhard, A. S. Bazanella, C. R. Rojas, and H. Hjalmarsson. 2012. On the convergence of the Prediction Error Method to its global minimum. *IFAC Proceedings Volumes* 45, 16 (2012), 698–703. doi:10.3182/20120711-3-BE-2027.00371

D. Eckhard, A. S. Bazanella, C. R. Rojas, and H. Hjalmarsson. 2013. Input design as a tool to improve the convergence of PEM. *Automatica* 49, 11 (2013), 3282–3291. doi:10.1016/j.automatica.2013.08.027

D. Eckhard, A. S. Bazanella, C. R. Rojas, and H. Hjalmarsson. 2017. Cost function shaping of the output error criterion. *Automatica* 76 (2017), 53–60. doi:10.1016/j.automatica.2016.10.015

Sawyer Fuller, Ben Greiner, Jason Moore, Richard M. Murray, René van Paassen, and Rory Yorke. 2021. The Python Control Systems Library (python-control). In *2021 60th IEEE Conference on Decision and Control (CDC)*. IEEE, Austin, TX, USA, 4875–4881. doi:10.1109/CDC45484.2021.9683368

G. C. Goodwin, J. C. Agüero, and R. Skelton. 2003. Conditions for local convergence of maximum likelihood estimation for ARMAX models. In *Proc. 13th IFAC Symposium on System Identification*. IFAC, Rotterdam, The Netherlands, 797–802.

Wilson Rocha Lacerda, Luan Pascoal Costa da Andrade, Samuel Carlos Pessoa Oliveira, and Samir Angelo Milani Martins. 2020. SysIdentPy: A Python package for System Identification using NARMAX models. *Journal of Open Source Software* 5, 54 (2020), 2384. doi:10.21105/joss.02384

L. Ljung. 1978. Convergence analysis of parametric identification methods. *IEEE Trans. Automat. Control* 23, 5 (1978), 770–783. doi:10.1109/TAC.1978.1101840

Lennart Ljung. 1999. *System Identification: Theory for the User* (2 ed.). Prentice Hall, Upper Saddle River, NJ.

L. Ljung. 2010. Perspectives on system identification. *Annual Reviews in Control* 34, 1 (2010), 1–12. doi:10.1016/j.arcontrol.2009.12.001

T. Söderström and P. Stoica. 1982. Some properties of the output error method. *Automatica* 18, 1 (1982), 93–99. doi:10.1016/0005-1098(82)90031-0

Torsten Söderström and Petre Stoica. 1989. *System Identification*. Prentice Hall, Englewood Cliffs, NJ.

K. Steiglitz and L. E. McBride. 1965. A technique for the identification of linear systems. *IEEE Trans. Automat. Control* 10, 4 (1965), 461–464. doi:10.1109/TAC.1965.1098181

P. Stoica and T. Söderström. 1981. The Steiglitz–McBride identification algorithm revisited—Convergence analysis and accuracy aspects. *IEEE Trans. Automat. Control* 26, 3 (1981), 712–717. doi:10.1109/TAC.1981.1102679

The MathWorks, Inc. 2026. System Identification Toolbox. Online. https://www.mathworks.com/products/sysid.html Accessed: 2026-05-25.